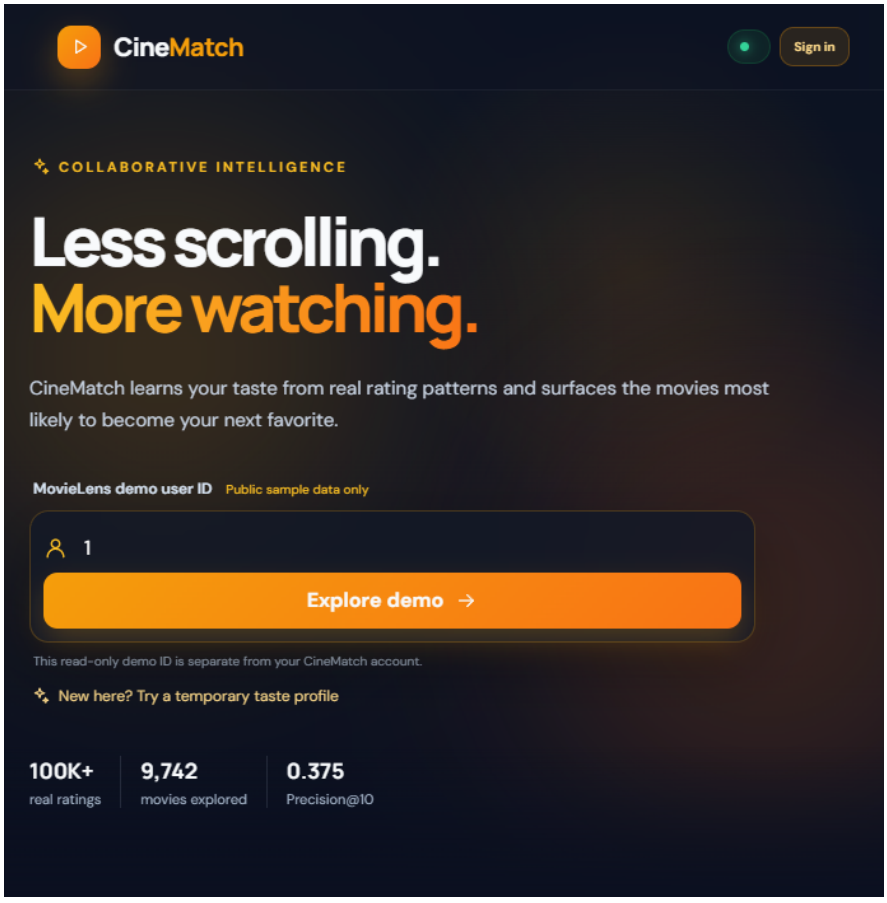
CineMatch

Collaborative Filtering Movie Recommendation System

Internship Project Report | Local implementation and offline evaluation



CineMatch is a local React and FastAPI application for exploring movie discovery from MovieLens ratings. This report describes the implemented collaborative-filtering experiments, saved SVD model, web integration, verification, limitations, and reproducible evidence.

Prepared 2026-11-07 - Not deployed to a public website

1. Executive summary

The project compares user-user collaborative filtering, item-item collaborative filtering, and Surprise SVD on MovieLens Small. Item-Item achieved the lowest rating prediction error on the shared 2,000-rating test sample (RMSE 0.8740; MAE 0.6617). SVD achieved the strongest Top-10 ranking results (Precision@10 0.375; NDCG@10 0.497) under the saved sampled-candidate protocol, so it powers demo discovery. New account profiles use a Bayesian-weighted popularity fallback until there are at least three ratings.

The implementation consists of a React/Vite frontend, FastAPI routes and recommendation services, MovieLens CSV assets, a saved 20-factor SVD model, and local SQLite persistence for accounts and their activity. The final project verification recorded 132 backend tests and 15 frontend tests passing, a passing production build, and a browser smoke test using a disposable account database.

Evidence	Observed result
Dataset	100,836 ratings - 610 users - 9,742 catalog movies
Rating comparison	Item-Item lowest RMSE: 0.8740
Ranking comparison	SVD highest Precision@10: 0.375 and NDCG@10: 0.497
Local verification	132 backend + 15 frontend tests passed; build passed
Hosting	No public URL verified; no deployment performed

2. Problem statement and objectives

Large movie catalogs create discovery overload. A popularity list can surface broadly rated titles but does not personalize the ordering. CineMatch investigates whether historical explicit ratings can support individualized unseen-movie ranking while retaining a public, read-only MovieLens demonstration.

- Explore rating distributions and sparsity in MovieLens Small.
- Implement and compare user-neighbor, item-neighbor, and matrix-factorization methods.
- Evaluate both rating prediction and ranked discovery with shared, seeded data splits.
- Integrate recommendations into a React interface backed by FastAPI.
- Keep public MovieLens demo behavior separate from private authenticated account records.

3. Data source and exploratory analysis

The included MovieLens Small snapshot provides 100,836 explicit ratings from 610 users over 9,742 catalog movies (9,724 movies have ratings). The user-by-rated-movie matrix has 5,931,640 possible cells, of which 100,836 are observed: approximately 98.3000% sparsity. The mean rating is 3.5016 and the median is 3.5; 39.70% of ratings are 4 or 5.

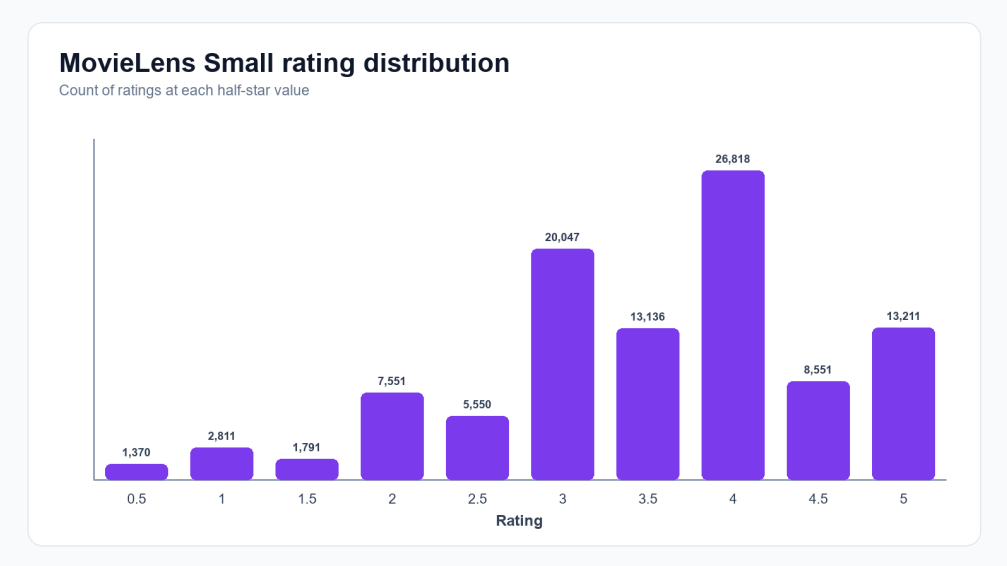


Figure 1. Distribution of explicit ratings in the MovieLens Small snapshot.

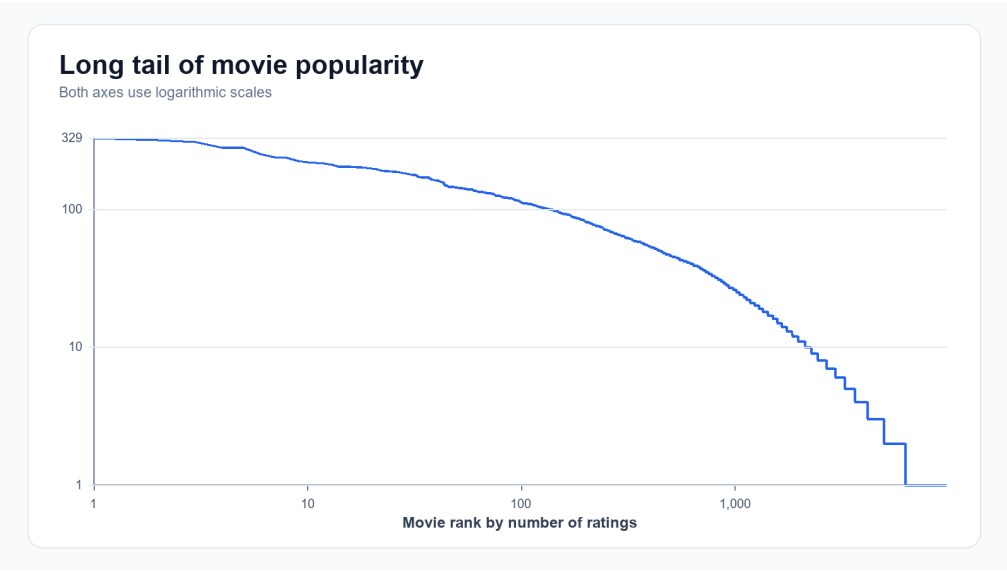


Figure 2. Long-tail behavior: rating evidence is concentrated on a subset of movies.

The high sparsity motivates fallback predictions and explicit coverage checks. Long-tail activity also creates a trade-off: recommendations can favor popular items with more evidence and provide less exposure to rarely rated titles.

EDA field	Value
Observed ratings	100,836
Users / catalog movies	610 / 9,742
Movies with ratings	9,724
Matrix density / sparsity	1.7000% / 98.3000%
Mean / median rating	3.5016 / 3.5
Share rated 4 or 5	39.6971%

4. Data preparation and experimental design

The evaluation pipeline uses a deterministic per-user 80/20 holdout with seed 42. It produces 80,672 training ratings and 20,164 held-out ratings. Rating metrics are computed on the same deterministic sample of 2,000 held-out ratings for each model, which supports a direct comparison on shared cases.

For ranking, 100 seeded users are evaluated at K=10. A relevant item is a held-out movie rated at least 4.0. Each user's candidate set contains held-out movies plus 100 sampled unseen movies. An unrated candidate is not a confirmed negative; therefore ranking metrics describe performance under this sampled-candidate protocol and should not be read as observed user dislike.

Protocol	Configuration
Rating split	Per-user 80/20; seed 42
Rating comparison sample	2,000 shared held-out ratings
Ranking users / cutoff	100 seeded users / K=10
Relevant threshold	Held-out explicit rating >= 4.0
Sampled unseen candidates	100 per user
SVD validation	12.5% of outer training split; seed 43

5. Recommendation algorithms

5.1 User-user collaborative filtering

This memory-based method compares users by the overlap and similarity of their rating vectors. For an unrated target movie, ratings from similar users are combined to estimate the target user's preference. The project uses mean-centered cosine similarity, up to 30 neighbors, and requires at least three common ratings. When there is insufficient neighbor evidence, the evaluation implementation uses a fallback prediction.

5.2 Item-item collaborative filtering

Item-Item computes relationships between movies from user rating patterns. A target user's ratings on related movies contribute to predictions for an unseen movie. The configured neighbor and overlap constraints match the User-User experiment, allowing a controlled comparison of the neighborhood direction.

5.3 Singular Value Decomposition

The project uses the SVD estimator from scikit-surprise 1.1.5. It learns compact latent-factor representations from observed ratings and estimates user-item preference from global, user, item, and latent interaction terms. Candidate factor counts were 20, 50, and 100. Validation selected 20 factors (validation RMSE 0.8779); the selected configuration uses 20 epochs.

5.4 Cold start and account personalization

For a new account without enough ratings, CineMatch returns a Bayesian-weighted popularity list with prior weight 50. For accounts with at least three saved ratings, the service folds account ratings into the SVD item-factor space and ranks unseen movies. Public MovieLens demo routes use the saved sample model and do not write to account records.

6. Rating prediction results

RMSE measures the squared-error penalty and MAE reports mean absolute error; lower values are better. All three models use the same 2,000 held-out ratings.

Model	RMSE (lower)	MAE (lower)	Interpretation
User-User	0.9025	0.6873	Higher error in this experiment
Item-Item	0.8740	0.6617	Lowest RMSE and MAE
SVD	0.8922	0.6835	Between the neighborhood models

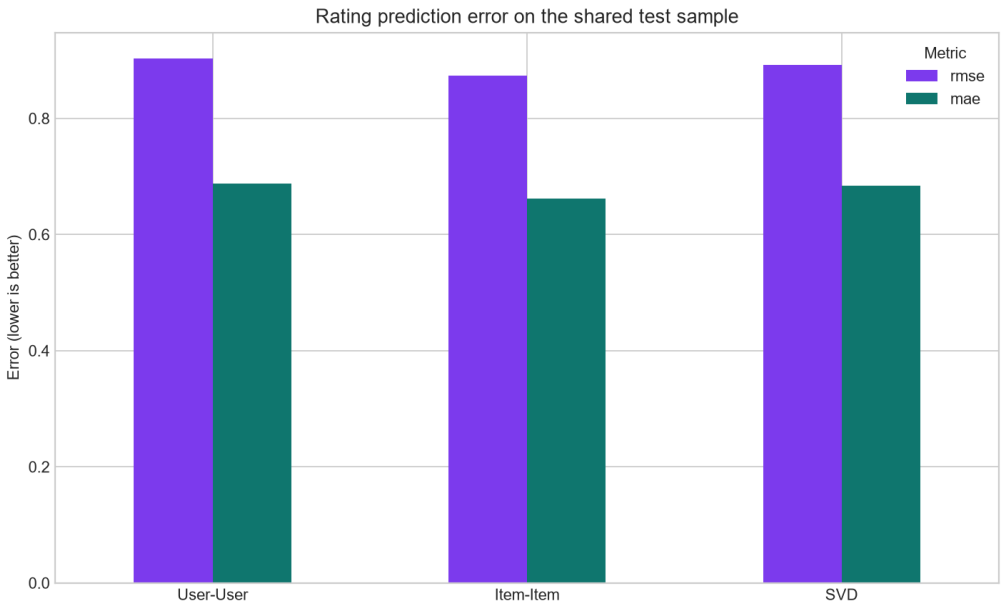


Figure 3. RMSE and MAE comparison on the common test sample.

Item-Item is the best model for estimating held-out ratings in this test. That result does not by itself determine the best ranked-discovery model, so ranking metrics are evaluated separately.

7. Top-K ranking results

Model	Precision@10	Recall@10	Hit Rate@10	NDCG@10	Coverage@10
User-User	0.122	0.115	0.650	0.1274	0.0781
Item-Item	0.309	0.316	0.880	0.3429	0.0673
SVD	0.375	0.369	0.890	0.4970	0.0520

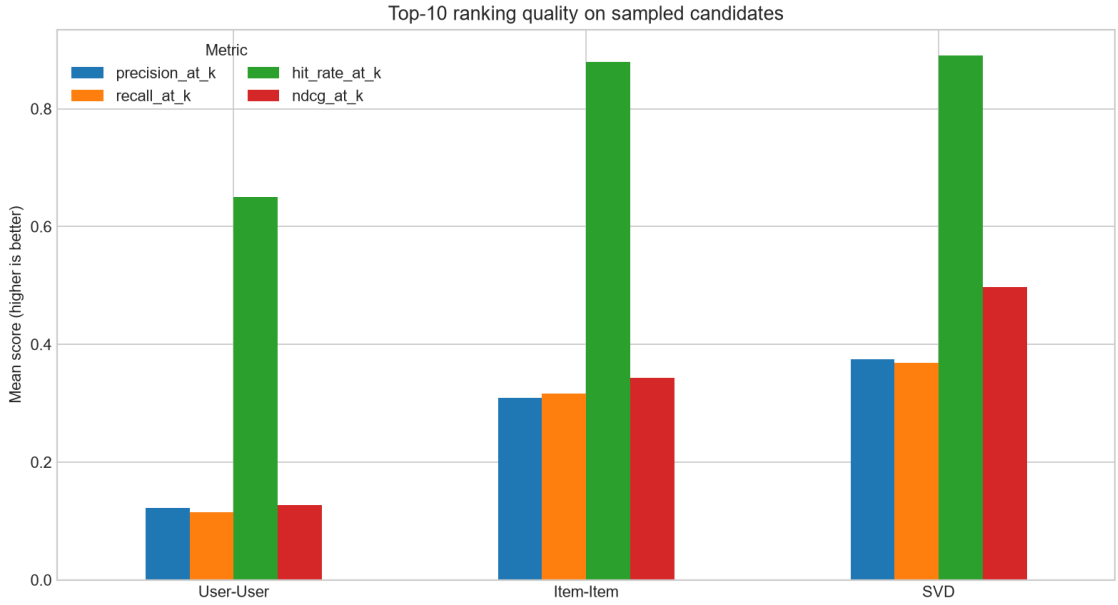


Figure 4. Top-10 ranking metrics under the shared sampled-candidate protocol.

SVD leads Precision@10, Recall@10, Hit Rate@10, and NDCG@10 in this evaluation. It also has the lowest catalog coverage and recommends fewer unique items than User-User (507 versus 761). The product selects SVD because its primary objective is relevant ranked discovery, not because SVD is universally superior.

8. Application architecture

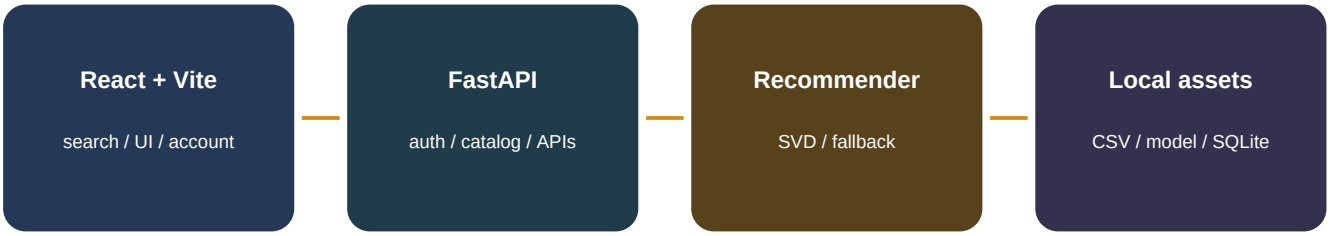


Figure 5. Logical local architecture. The frontend and backend run separately in development; a built frontend can be served by FastAPI.

Frontend: React and Vite provide movie search, genre filters, movie detail views, onboarding, account screens, and recommendation panels. During local development, the Vite server proxies /api/v1 requests to FastAPI.

Backend: FastAPI exposes health, catalog/search, similar-movie, prediction, recommendation, authentication, and account-action endpoints. The saved SVD model is loaded for demo inference; known MovieLens users receive unseen-title rankings. Cold-start account profiles use the Bayesian-weighted popularity fallback.

Persistence: SQLAlchemy and SQLite store local account, session, rating, watchlist, and feedback data. Alembic manages the local authentication schema. The real local database was preserved during final submission verification; browser account flow used a disposable test database.

9. Product and security boundaries

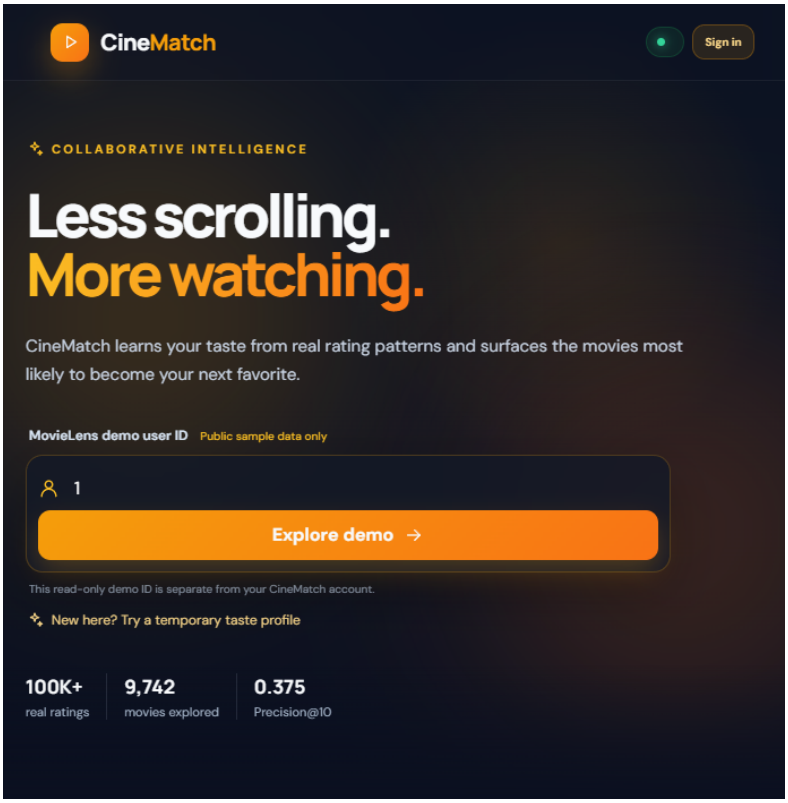


Figure 6. Logged-out CineMatch homepage with a clearly labeled public MovieLens demo.

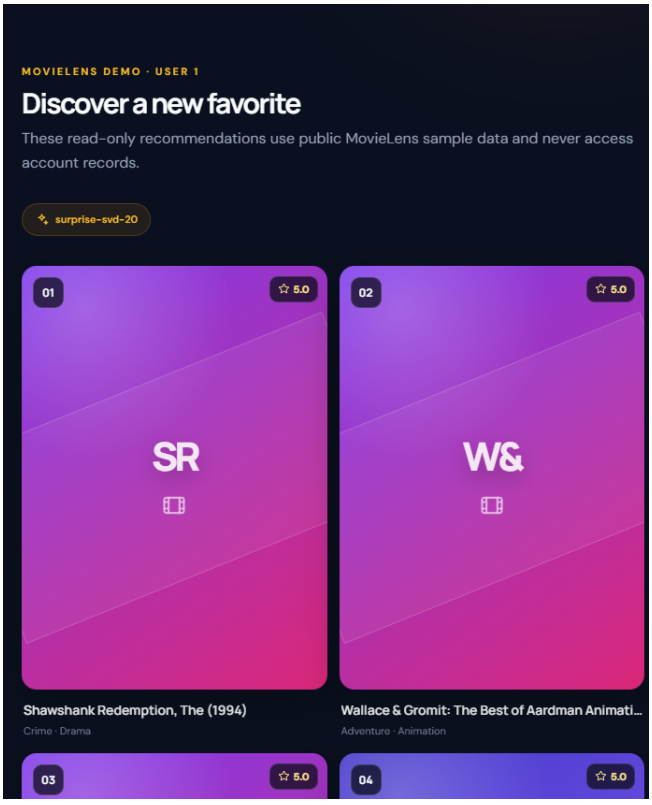


Figure 7. Read-only MovieLens demo recommendations are labeled separately from account data.

Authentication uses registration/login/logout/current-user routes and server-managed HttpOnly session cookies. After login, the session identity rather than a user-editable numeric ID selects private account recommendations and actions. Logged-out users can still browse the public catalog and read-only MovieLens demo. Backend authorization remains

the enforcement boundary.

No credential values are included in this report. The live frontend/API integration is local; the report does not claim that a public deployment exists.

10. Verification and reproducibility

Check	Recorded result
Backend suite	python -m pytest -q: 132 passed; 2 deprecation warnings
Frontend tests	pnpm test: 15 passed
Frontend build	pnpm build: passed with Node 24.22.1
Browser/API smoke	Local public browsing and disposable-account workflow passed
Database safety	Real database preserved; no real migration run
Deployment	Not deployed; no public URL verified

The browser/API smoke test covered homepage, movie search/details, public MovieLens demo, registration, login, session-backed personalized recommendations, three ratings, watchlist, feedback, and logout. The test account was created against a disposable database.

To reproduce locally, follow the PowerShell installation and startup steps in README.md. Use Python 3.12, install backend requirements, install frontend packages from the lockfile with pnpm 9.15.0, start FastAPI and Vite in separate terminals, and run pytest plus frontend tests.

11. Limitations and future work

- MovieLens Small is a historical research/development snapshot, not a live catalog.
- Offline ranking metrics do not measure satisfaction, novelty, diversity, fairness, or long-term use.
- Sampled unrated movies are not known negatives; results depend on the sampled-candidate protocol.
- SVD factors are not inherently interpretable as genres or explanations.
- New accounts depend on the popularity fallback until sufficient rating history exists.
- SQLite and process-local authentication rate limiting are suitable for the local demo, not horizontally scaled production.
- Generated movie artwork is used rather than a live poster service.

Practical next steps

- Evaluate temporal splits and test more users and candidate sets.
- Measure recommendation diversity and catalog coverage alongside relevance.
- Run consented user studies before drawing claims about user satisfaction.
- Only if deployment is approved, establish durable database storage and shared rate limiting first.

12. Conclusion

CineMatch demonstrates an end-to-end recommender workflow: exploratory analysis of sparse ratings, controlled comparisons of memory-based collaborative filtering and SVD, offline rating and ranking evaluation, and local web integration. The evidence supports a task-specific choice: Item-Item is strongest for rating error while SVD is strongest for Top-10 ranking in the measured protocol. The implemented product combines SVD discovery with a cold-start fallback and keeps public demo behavior distinct from account features.

References

- GroupLens Research. MovieLens datasets. https://grouplens.org/datasets/movielens/ (dataset attribution and license are included with the project data).
- Harper, F. M., & Konstan, J. A. (2015). The MovieLens Datasets: History and Context. ACM Transactions on Interactive Intelligent Systems, 5(4). https://doi.org/10.1145/2827872.
- Koren, Y., Bell, R., & Volinsky, C. (2009). Matrix Factorization Techniques for Recommender Systems. Computer, 42(8), 30-37. https://doi.org/10.1109/MC.2009.263.
- Surprise documentation. Matrix factorization-based algorithms, SVD. https://surprise.readthedocs.io/en/stable/matrix_factorization.html.
- CineMatch project evidence: data/processed/week1_summary.json, week2_summary.json, week3_summary.json, week4_summary.json; reports/week4_evaluation_report.md; README.md.

Submission verification update

Addendum dated 2026-11-07 - supplements the main report.

Updated test results

The main report records the initial 132-test backend result. After adding five read-only-demo configuration and write-guard regressions, the complete backend suite was rerun: **137 passed, 2 deprecation warnings**. Frontend verification: **15 tests passed**; the Vite production build passed on Node 24.22.1 with the read-only-demo flag enabled.

Read-only demo preparation

The deployment configuration now uses an isolated SQLite file under /tmp, rejects production read-only SQLite paths outside /tmp, blocks account/session/data writes, and keeps only stateless onboarding recommendations available. The Docker frontend build hides sign-in prompts and labels the public demo. Local account features remain enabled in normal development mode.

Public URL status

The user approved publishing a sanitized public source repository and a Render Free read-only demo. No repository or hosting service was created: GitHub navigation failed and Render navigation timed out in this environment, and no authenticated GitHub/Render CLI session is available. The project is **not deployed**; the bundled LIVE_WEBSITE_URL.txt explicitly says that no URL is verified.

Deployment limitation	Render Free behavior
Storage	Ephemeral; demo data and SQLite file are disposable
Availability	Service can sleep after inactivity and take about a minute to wake
Account features	Disabled; no persistent account data